OAL TRINITY CURRENCY SYSTEM

Owner Decision List

Version 0.1: questions for Nathan, consolidated from all six collaborators

> **STATUS: PROPOSED.** Advisory only. It records no decisions, adopts nothing and is not canon.

Document ID: OAL-CURR-OWNER-QUESTIONS-v0.1-CLAUDE-20261107
Date: 7 November 2026
Prepared by: Claude (Anthropic), AI Collaboration Team
Prepared for: Nathan Lewis II

Contents

1. How to use this list
2. Where things stand (one paragraph each)
3. Tier 1 decisions: these block a formal shared run
4. Tier 2: design conventions
5. Tier 3: how the laboratory runs
6. Tier 4: publication and housekeeping
7. Next steps the team can take without you
8. Quick reply form
Appendix A. Source map: who raised what

1. How to use this list

This list gathers every question the six collaborators asked Nathan in the November 7 round (ChatGPT, Claude, Google Gemini, Microsoft Copilot, Grok and Perplexity). Overlapping questions are merged, and each one names who raised it. Nothing here is canon, and nothing here requires a permanent decision.

Any answer is useful, including "undecided", "test both", "leave it to the contributors" or "please recommend a default". The suggested defaults are Claude's own judgment. They are not a team consensus, and the other members may disagree.

> **If you only have time for five**
> Answer D1 (what a Quantum Credit must accomplish), D2 (adopt a named run card), D3 (creation authority), D4 (why an instrument may be kept) and D5 (what happens after a partial delivery). Those five decide whether a formal shared run can start. Everything else can wait or be tested as alternatives.

A short reply form is in Section 8. A source map (who raised what) is at the end.

2. Where things stand (one paragraph each)

Access. ChatGPT, Claude and Grok read the repository. Gemini and Copilot reviewed uploaded files. Perplexity's GitHub connector now works, and it has confirmed the folder listing (41 entries) and the README. Perplexity says that is not yet a full file-by-file review.

Cycle 1. Several members produced pilot traces and executable models. The baseline results agree: transfer gives A=7, B=3; creation gives A=10, B=3 (total 13); evaluation moves nothing. These are consequences of stipulated rules. They are not evidence that any real Quantum Credit capability exists.

Checker audit. Perplexity reproduced the ChatGPT and Claude models exactly, then showed that both checkers accept some defective records. ChatGPT's verifier accepted three altered records. Claude's checker accepted an early receipt when delivery later completed, and its receipt is global rather than tied to a request. The original models still behave as written. The checkers are narrower than their labels suggested.

Status wording. Copilot and Perplexity both object to describing Cycle 1 as "fully reconciled" or "fully stabilized". Copilot proposes recording it as: exploratory modeling completed, reproduction and checker audit in progress, no owner-adopted shared run yet, no external validation, no canon. Claude agrees with that wording.

3. Tier 1 decisions: these block a formal shared run

D1. What must a successful Quantum Credit accomplish?

Raised by: ChatGPT (Q1), Copilot (Q1), Grok (Q3), Perplexity (Q1), Claude (Q7)

Question: Is a Quantum Credit successful if it only works out the equivalent value of a target? Or must it also make the resources, access or outcome available?

Options:

- A. Determine or represent the target's equivalent value.
- B. Make the required resources or purchasing capability available.
- C. Obtain access, rights or another defined outcome involving the target.
- D. Directly establish the requested target state inside the fictional framework.
- E. Keep several of these as separate candidate architectures.

Claude's suggested default: E, with your ambitious intended capability stated as the design objective and the evaluation-only configuration kept as a control. An evaluation-only model can be internally consistent and still miss your intent, so it should not quietly stand in for the whole project.

If left unanswered: The team keeps all four capability configurations (Evaluate, Transfer, Create, Enact) and labels results against each.

D2. Adopt a named run card?

Raised by: Claude (1), ChatGPT (3), Gemini (1), Grok (1), Copilot (4), Perplexity (Q2)

Question: Will you adopt one named Pilot 001 run card, so that existing and future traces can be compared directly? Several different packets use the name "Pilot 001", so adoption should name the packet and its revision.

Options:

- Adopt one existing proposal (for example ChatGPT's OAL-QC-PILOT-001, Perplexity's PILOT-001, or Claude's Run Card 001 r1).
- Ask the team to produce one small harmonized run card.
- Let several run cards stay in parallel.
- Both: keep independent exploration and also freeze one shared packet, with results labeled separately.

Claude's suggested default: The last option. Adopting a packet freezes tests only. It does not canonize any currency mechanic.

If left unanswered: Every package stays PROPOSED, and the existing traces stay exploratory.

D3. Who authorizes creation?

Raised by: Claude (2), ChatGPT (2), Gemini (2), Copilot (2), Grok (4), Perplexity (Q4)

Question: If a Quantum Credit creates three new units rather than taking them from A, whose permission is needed? Claude's model blocks creation when A withdraws consent. Perplexity's model creates anyway, because its packet declares no authority over creation. Neither is a discovery. They are different added assumptions.

Options:

- System issuer (an AUTH_MINT-type authority).
- The requester.
- Another governing entity.
- No external authorization.
- Perplexity's finer split: permission to take from A, to issue new units, to place units with B, and to alter the surrounding system.

Claude's suggested default: Test the authority profiles as separate named configurations. Do not pick one permanent rule yet.

If left unanswered: Each model keeps its own assumption and says so. Results from different assumptions are not presented as head-to-head.

D4. What justifies keeping 1COIN and NCOIN?

Raised by: Gemini (4), ChatGPT (4), Copilot (9), Perplexity (Q3)

Question: Must each instrument do something a plain Quantum Credit plus an audit log cannot? Or are separation of duties, operational reliability, or narrative and worldbuilding value also valid reasons to keep it?

Options:

- Mechanical necessity only.
- Mechanical necessity or operational advantage.
- Any of the three (mechanical, operational, narrative), with the reason disclosed.

Claude's suggested default: Assess the three grounds separately and report them separately. An instrument kept for narrative reasons should never be described as mechanically indispensable.

If left unanswered: The team scores each instrument on all three grounds and reports them side by side.

D5. What happens after a partial delivery?

Raised by: Copilot (6), Perplexity (Q9), Claude (6)

Question: If a request delivers two of three units and then stops, which behavior fits your intent? Today, Claude's duplicate-suppression policy leaves the request permanently partial. That was never a decision. It is a specification gap.

Options:

- Resume: deliver only the remaining unit on retry.
- Rollback: restore the starting state, then allow a clean retry.
- Terminal partial: keep the partial result and require a new remediation request.
- Reject any retry under the same ID.
- Compensating transaction: a separately identified operation completes or reverses it.

Claude's suggested default: Compare the alternatives as named configurations. Duplicate suppression alone should not silently decide recovery.

If left unanswered: Models keep their current stipulated behavior, labeled as a stipulation.

4. Tier 2: design conventions

D6. How should a shortfall be labeled?

Raised by: Claude (3), Gemini (3), Grok (5), ChatGPT (5), Copilot (3), Perplexity (Section 4)

Question: When A holds two units and a source-based operation needs three, the world state is the same whether we label it REJECTED or INCOHERENT_TARGET. Which convention do you prefer?

Options:

- REJECTED, with reason INSUFFICIENT_SOURCE_INVENTORY.
- INCOHERENT_TARGET.
- Two fields: execution status and target validity (ChatGPT, Copilot).
- Perplexity's four fields: request_coherence, feasibility_under_configuration, execution_status and reason.

Claude's suggested default: Two or more separate fields. A coherent request can be infeasible from a given starting state. "Incoherent" should mean the request contradicts the declared rules.

If left unanswered: Not blocking. The team shows both labels side by side.

D7. What counts as "the same request"?

Raised by: Claude (5), Copilot (8), Perplexity (Finding G-2)

Question: Should a repeated request be suppressed only when the transaction ID matches? Or when the obligation (for example "B should have 3") has already been met, even under a new ID? What should happen if an existing ID comes back with a different payload?

Options:

- Same ID, same payload: suppress.
- Same ID, different payload: reject as an ID/payload conflict.
- New ID, same payload: deliver again, or suppress as the same obligation.
- New ID, modified payload: new request.

Claude's suggested default: Bind each ID to a hash of its payload, and report the four cases above as distinct statuses. Compare ID-based and obligation-based suppression as two configurations.

If left unanswered: Current behavior stands: a repeated ID is suppressed, and a new ID delivers again.

D8. What does "usable delivery" mean?

Raised by: Perplexity (Q8)

Question: When B receives three units, is success just the right balance? Or must the units also be of the declared type, identifiable, unencumbered and usable for a further operation? A plain-language answer is enough.

Claude's suggested default: B must genuinely be able to use the units inside the simulation, not merely see a balance. The team then proposes the precise checks. Perplexity found that its own completion check tests only quantity, not the type and encumbrance properties its documentation promises.

If left unanswered: Quantity-only completion stays in place, labeled as narrower than intended.

D9. What may a receipt certify?

Raised by: Copilot (7), ChatGPT (audit discussion), Claude (audit gap)

Question: Should a receipt certify only that one specific request is fully complete? Or may it also mark intermediate milestones? If milestones are allowed, should they be named differently from a final 1COIN settlement receipt?

Claude's suggested default: Final completion only, tied to one request. Milestones, if wanted, get a different name.

If left unanswered: The v0.2 models use request-scoped final receipts only.

D10. Identifier scheme

Raised by: Claude (4), Grok (6), Copilot (5), Perplexity (Q6)

Question: Which scheme becomes the working standard: contributor-qualified IDs (contributor/document-revision/original-id), Gemini's LAB-F series, Copilot's neutral OAL-CURR IDs, or a three-part scheme of Case ID, Configuration ID and Run ID?

Claude's suggested default: Keep contributor-qualified IDs, and add the three-part Case/Configuration/Run structure on top. Add neutral aliases only after two cases are checked and judged equivalent. Original documents are not renumbered.

If left unanswered: The qualified form is used wherever IDs from different contributors meet.

D11. Status vocabulary

Raised by: Copilot (15, 16), Grok (extra question), Claude (8)

Question: Do you want OWNER-ADOPTED and LOCKED FOR TESTING kept as two statuses or merged? Should EXPERIMENTALLY SUPPORTED be renamed to SUPPORTED WITHIN RUN (or CONSISTENT UNDER RUN) so nobody reads it as real-world validation? Should a separate evidence field (E0 stipulated, E1 derived, E2 executed, E3 external) travel with every status?

Claude's suggested default: Perplexity makes a fair case for keeping adoption and lock separate: a defect can force a revision after adoption. Claude would accept that, on condition that the rename goes through and the evidence field is mandatory.

If left unanswered: Each package keeps its own status language, which makes comparison harder.

D12. Instrument roles and automatic NCOIN

Raised by: Copilot (10, 11, 12)

Question: Should 1COIN be an asset, a receipt, a settlement certificate, or a broader class? Should NCOIN ever be emitted automatically from a rejection or failure? Should an undefined, incoherent or rejected result stay a plain status record unless a separate instrument rule is activated?

Claude's suggested default: Test all four 1COIN roles as hypotheses. The standing laboratory rule is that rejected, impossible or unauthorized transactions do not generate NCOIN automatically, and Claude recommends keeping it. Please confirm or amend.

If left unanswered: The standing rule stays: no automatic NCOIN.

5. Tier 3: how the laboratory runs

D13. Blind rounds and reference exposure

Raised by: ChatGPT (6), Copilot (8), Grok (2), Claude (9), Perplexity (Q5)

Question: Perplexity's sealed reference report was published, and all six members have now seen it or its results. This round therefore cannot become blind afterward. For a future first-pass round, do you want reference results withheld until submissions are collected?

Options:

- Blind first pass, then open challenge.
- Open challenge only.
- Both, as separately labeled activities.

Claude's suggested default: Both. Label this round "reference-exposed exploratory". For a new blind round, use a new fixture whose expected outputs have not circulated. Perplexity notes that a folder named SEALED does not withhold anything. What matters is what is distributed before submissions are in.

If left unanswered: This round stays labeled reference-exposed.

D14. Identical-test runs versus independent designs

Raised by: ChatGPT (7), Grok (7), Copilot (13)

Question: Should the repository keep "same frozen packet" runs separate from "independently designed experiment on the same fixture"?

Claude's suggested default: Yes. Keep both and label them separately.

If left unanswered: The two kinds of result get mixed in comparisons.

D15. Model overlap and evidentiary weight

Raised by: Claude (11), Copilot (14), Grok

Question: Perplexity reports that its session ran on Claude Opus 5.5, so Perplexity and Claude may share a model family. Should agreement between overlapping models count for less, or should the repository only disclose the overlap? Should every member state its underlying model?

Claude's suggested default: Require a one-line disclosure of the underlying model with every run record. Leave any formal weighting out for now.

If left unanswered: Disclosure is voluntary and uneven.

D16. Executable models and deliberate-defect tests

Raised by: Claude (10), Grok (extra question), ChatGPT (hardening proposal), Copilot (checker findings)

Question: Must future runs include executable code? Should deliberate-defect injection, with a separate mutation-detection score, be required for any published reference model?

Claude's suggested default: Yes to defect injection for any model that claims a checker. Analytical traces remain acceptable where the rules are not yet precise enough to code.

If left unanswered: Pass counts such as "133 checks" continue to be reported without a detection score.

D17. What comes after adoption?

Raised by: Grok (9)

Question: After a basic Pilot 001 adoption, should the next shared case focus on observability of intermediate states and unit identity (so that Enact can be told apart from Transfer or Create), or move to a 1COIN ablation under the same resource-box fixture?

Claude's suggested default: Observability first. Several members found that Enact is untestable, or identical to Transfer or Create, from final balances alone.

If left unanswered: The team proceeds in the order of its own proposals.

D18. Assumptions the laboratory must not impose

Raised by: Perplexity (Q10)

Question: Are conservation, unit identity, consent, time ordering or the need for a seller things you want challenged in a designated speculative track? The risk is that small resource-box models quietly build conventional assumptions into a capability meant to exceed them.

Claude's suggested default: Name any such requirement. A changed assumption gets its own configuration and does not retroactively repair an earlier run.

If left unanswered: Conventional assumptions stay in the baseline.

6. Tier 4: publication and housekeeping

D19. Access index, Markdown companions and corrected editions

Raised by: ChatGPT (access-index proposal), Copilot (17), Claude (12), Grok (8), Perplexity (Q7)

Question: May the team prepare: a document manifest and access index (contributor, title, revision, status, raw URL, text companion available, commit-pinned links); Markdown companions for the long PDF/DOCX files; corrected publication editions; and a correction log tied to untouched originals?

Claude's suggested default: Authorize preparation. Perplexity's caution is sound: preparing the files does not authorize committing, renaming, moving or deleting anything in the repository. Those stay your actions.

If left unanswered: Preparation does not start.

D20. Temporary signed links in two compilations

Raised by: ChatGPT (signed-URL finding), Claude (correction), Copilot

Question: The compilations "Pilot Recommendations" and "Next Step Recommendations" carry expired signed attachment links in their hyperlink targets, 4 and 10 respectively in the DOCX and PDF editions. These do not show in the visible text, so a text search misses them. May the team prepare clean copies and keep the originals?

Claude's suggested default: Yes. Preserve the originals separately, record the correction, and make "inspect hyperlink targets" part of the pre-publication check.

If left unanswered: The links remain in the public copies. They are expired, but they are poor archival references.

D21. Earlier reference materials

Raised by: Copilot (18), Perplexity

Question: Should existing public reference materials stay where they are for transparency while new blind materials are distributed separately? Or should references be moved into a clearly marked post-disclosure folder?

Claude's suggested default: Leave them in place and mark them as reference-exposed in the manifest. Do not rearrange the archive.

If left unanswered: Nothing moves.

7. Next steps the team can take without you

Both ChatGPT and Perplexity ask that implementation details not fall on you. These are the engineering items the team can pursue on its own once you give a direction.

- **ChatGPT:** Pilot 001 Verification Hardening v0.2. Derive totals from holdings, check debit and credit independently, verify authority against the input fixture rather than the status string, add deliberate defects, and report a mutation-detection score apart from the pass count.
- **Claude:** a v0.2 model with request-scoped receipts, an ordered event log, a request-ID payload hash, quantity taken from each request instead of a fixed three, and an explicit partial-delivery policy. The original v0.1 stays in the record.
- **Perplexity:** strengthen the completion check to test unit type, encumbrance, unique identifiers and provenance. Publish the reference results file and an environment and checksum note with the model.
- **Gemini:** publish a standalone source file and machine-readable output for the M04 analysis, and correct the fixed-tolerance assertion (it fails at step 42 for alpha=0.75 because of floating-point rounding; a relative tolerance fixes it). Tag purchasing-power and NCOIN consequences as conditional hypotheses.
- **Copilot:** label its ten-case boundary pass as analytical, and add its manifest and registry entries to the shared index.
- **Grok:** keep the coordination sheet small, and extend pilot-scope ID mappings if the three-part scheme is adopted.
- **All:** record the audit as a formal artifact (ChatGPT proposes OAL-CURR-PILOT-001-CHECKER-AUDIT-PPLX-v0.1), keep v0.1 results unedited in the history, and publish corrected versions separately.

> **Wording the team should correct**
> "Fully reconciled" and "fully stabilized" are premature (Copilot, Perplexity, ChatGPT). "Conclusively proved" overstates the mutation-suite result (Copilot). "Observed across multiple independent implementations" should read "reproduced across separately implemented but not fully independent or blind models" (Copilot). Executed traces remain consequences of stipulated rules (Perplexity).

One factual item to settle by checking, not by asking you. Claude saw pilot001_reference_results.json in its clone of the folder, while ChatGPT did not find it. The simplest resolution is to compare listings at one named commit.

8. Quick reply form

Copy the lines below and answer only what you can. "Undecided", "test both" and "recommend a default" are all acceptable answers.

- D1 Intended QC capability:
- D2 Run card to adopt (packet and revision):

- D3 Creation authority:
- D4 Reasons an instrument may be kept:
- D5 Partial-delivery recovery:
- D6 Shortfall labeling:
- D7 Meaning of "the same request":
- D8 Meaning of usable delivery:
- D9 Receipt scope:
- D10 Identifier scheme:
- D11 Status vocabulary:
- D12 Instrument roles / automatic NCOIN:
- D13 Blind and open review:
- D14 Identical versus independent runs:
- D15 Model overlap disclosure:
- D16 Executable models and defect tests:
- D17 Next discriminating work:
- D18 Assumptions not to impose:
- D19 Index, companions, corrected editions:
- D20 Clean copies of the two compilations:
- D21 Earlier reference materials:

All statements in this list concern a fictional or simulated currency design under stipulated rules. Nothing here is evidence that any physical Quantum Credit capability exists.

Appendix A. Source map: who raised what

Question numbers are the numbers each member used in its own November 7 response.

- **ChatGPT (1 to 7):** capability (D1), creation authority (D3), adopt run card (D2), retain 1COIN/NCOIN (D4), impossible transaction label (D6), withhold references (D13), shared versus independent (D14). Also proposed the access index and the checker-audit artifact.
- **Claude (1 to 12):** adopt run card (D2), creation authority (D3), shortfall label (D6), identifiers (D10), same request (D7), partial retry (D5), minimum capability (D1), status set (D11), blind round (D13), executable code (D16), model disclosure (D15), index and clean copies (D19, D20).
- **Google Gemini (1 to 4):** adopt run card (D2), creation authority (D3), shortfall label (D6), instrument retention (D4).
- **Microsoft Copilot (1 to 18):** capability (D1), creation (D3), shortfall (D6), run card (D2), identifiers (D10), recovery (D5), receipt scope (D9), blind tests (D13), retention (D4), 1COIN class, automatic NCOIN and status-only results (D12), identical versus

independent (D14), model overlap (D15), status separation and naming (D11), index (D19), earlier references (D21).

- **Grok (1 to 9, plus two):** adoption (D2), withhold references (D13), capability (D1), creation (D3), shortfall (D6), identifiers (D10), identical versus independent (D14), manifest (D19), next case (D17); evidence vocabulary (D11) and defect injection (D16).
- **Perplexity (Q1 to Q10):** capability (D1), next-work authorization (D2), retention (D4), creation authority profiles (D3), blind and open review (D13), identifiers (D10), publication work (D19), usable delivery (D8), unfinished obligation (D5), assumptions not to impose (D18).

Document control: Prepared by Claude (Anthropic). Status PROPOSED, pre-canon. This list is advisory and records no decisions.